

WellRing — Production Backend Architecture Blueprint

Scope: `backend/` repository design for the WellRing AI elder-care platform. **Runtime:** Python 3.12 · FastAPI · Pipecat AI v1.2.1+ · PostgreSQL 16 + pgvector · Redis · Celery **Date:** May 2026 · Treat all vendor pricing as of this date.

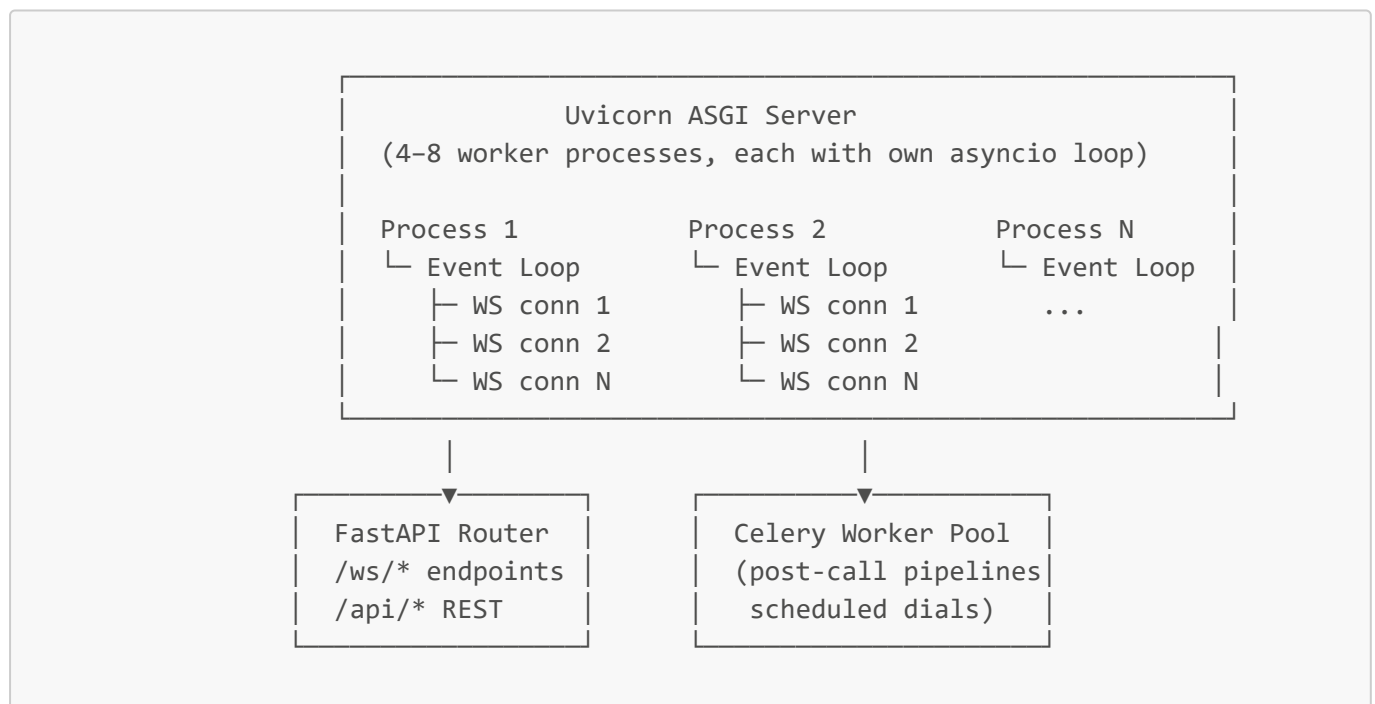
Table of Contents

1. [Core Framework & Concurrency Pipeline](#)
2. [Multi-Feature Pipeline Architecture & Model Matrix](#)
3. [Telephony & RAG Architecture](#)
4. [Production Directory Structure](#)
5. [Architectural Code Snippets](#)
6. [Appendix: Dependency Manifest](#)

1. Core Framework & Concurrency Pipeline

1.1 Foundational Runtime Model

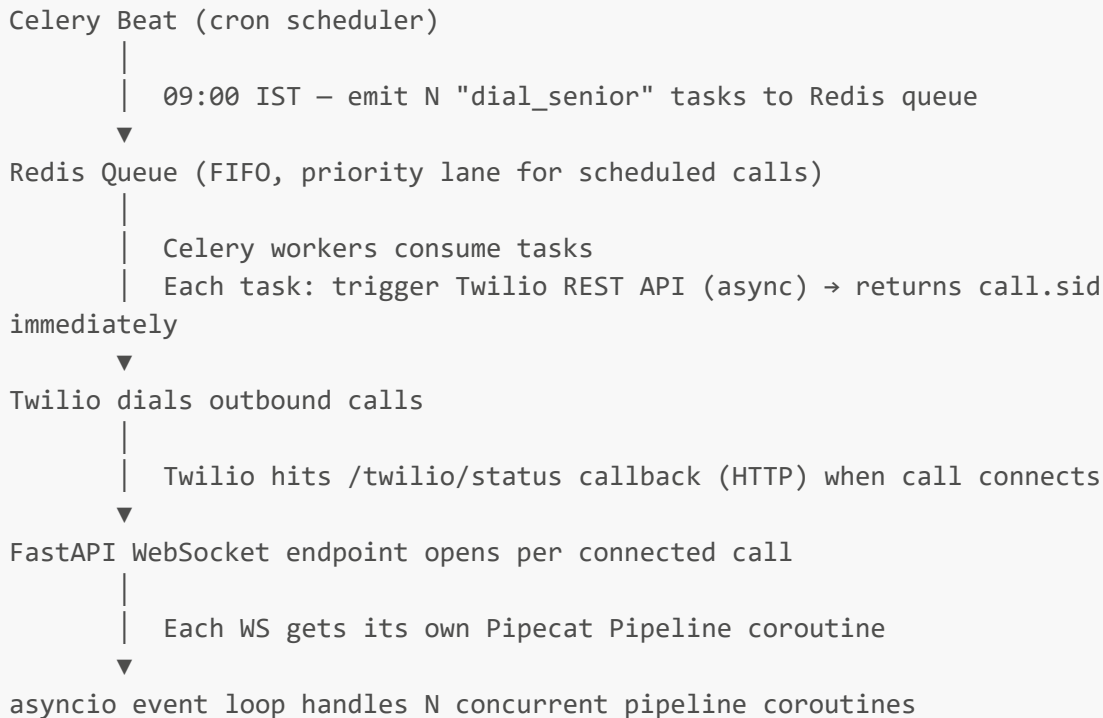
WellRing's backend is built on a **single-process asyncio event loop** (one Uvicorn worker per CPU core) horizontally scaled behind a load balancer. Every inbound WebSocket from Twilio and every outbound Pipecat pipeline runs as a coroutine — never a thread, never a subprocess — so thousands of concurrent calls share the same event loop without GIL contention.



Why this beats a thread-per-call model: A 5-minute voice call consumes ~\$0.15 in API costs but only ~2ms of actual CPU time (the rest is I/O-waiting on Deepgram, the LLM, and ElevenLabs). asyncio multiplexes 500 concurrent calls on 8 cores with ~\$120 in infra. An equivalent thread model requires ~500 OS threads, 4 GB of stack RAM, and a context-switch storm.

1.2 The 8:00 AM Concurrency Spike

When the scheduler fires 200 outbound calls simultaneously at 9:00 AM IST, the architecture must absorb that without queuing visible latency into the calls themselves. The strategy is a **two-stage fan-out**:



Key invariant: The Celery task does nothing except call `twilio.calls.create()` (an HTTP POST). No audio processing, no LLM calls, no blocking. Actual audio pipeline work only begins after Twilio connects and opens a WebSocket. This means 200 Celery tasks complete in under 2 seconds, and the audio pipelines spin up spread across a 10–30 second connection ramp as Twilio establishes PSTN circuits.

Celery configuration for this workload:

```

# workers/celery_app.py
from celery import Celery
from kombu import Queue

celery_app = Celery("wellring")
celery_app.config_from_object("core.celery_config")

# Dedicated queue for scheduled dial-outs – no starvation from post-call jobs
celery_app.conf.task_queues = (
    Queue("dial_out", routing_key="dial_out"),
    Queue("post_call", routing_key="post_call"),
    Queue("alerts", routing_key="alerts"),
)

celery_app.conf.task_routes = {
    "workers.tasks.dial_senior": {"queue": "dial_out"},
    "workers.tasks.run_post_call_pipeline": {"queue": "post_call"},
}
  
```

```

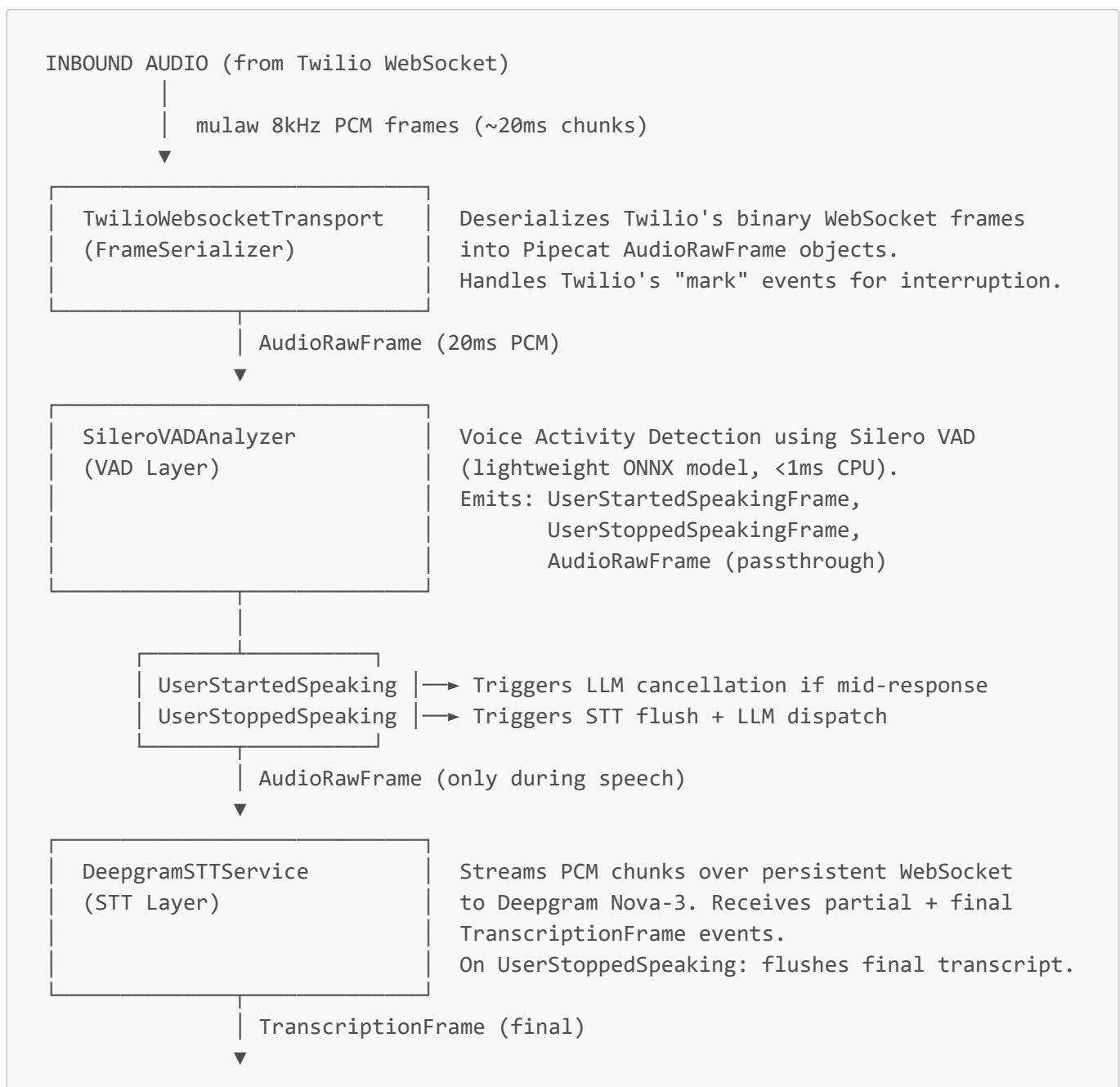
    "workers.tasks.dispatch_alert": {"queue": "alerts"},
}

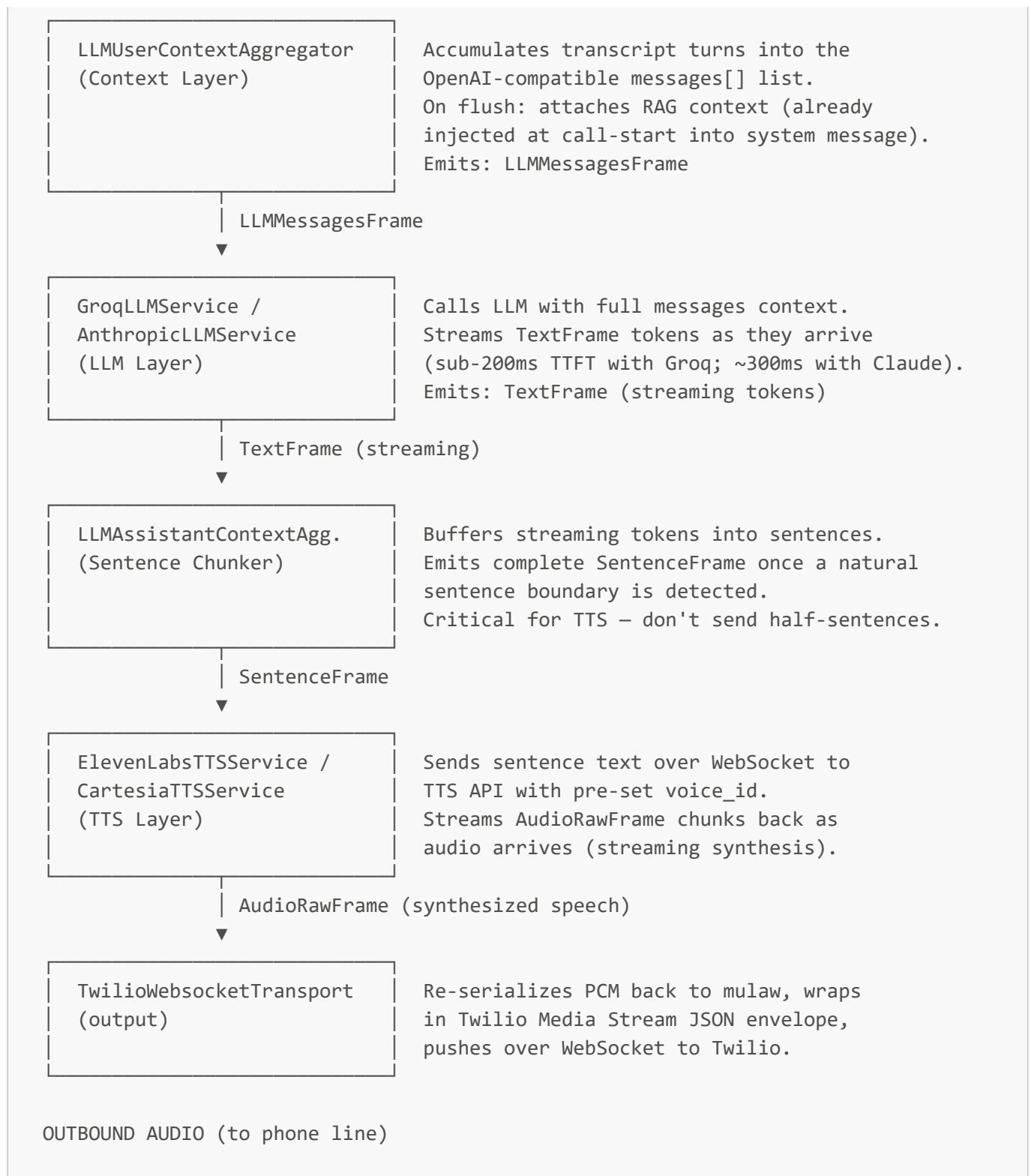
# Beat schedule
celery_app.conf.beat_schedule = {
    "morning-dial-sweep": {
        "task": "workers.tasks.morning_dial_sweep",
        "schedule": 60.0, # runs every minute; internal logic filters by
        scheduled time
    }
}

```

1.3 The Audio Streaming Lifecycle: Explicit Handoffs

This is the critical path. Every millisecond of added latency here degrades perceived call quality. The pipeline is a directed acyclic graph of Pipecat `FrameProcessor` objects connected by `asyncio.Queue` bridges.





Interruption handling is the trickiest part of the lifecycle. When `UserStartedSpeakingFrame` fires mid-TTS-playback:

1. `TwilioWebsocketTransport` sends a Twilio `clear` message to flush buffered audio on the PSTN side.
2. A `StopInterruptionFrame` propagates downstream, causing `ElevenLabsTTSService` to cancel its current synthesis request.
3. The `LLMUserContextAggregator` marks the interrupted assistant turn as partial in the conversation history.

This produces natural-feeling interruptions rather than the robot-finishing-its-sentence behavior.

2. Multi-Feature Pipeline Architecture & Model Matrix

2.1 Tier Architecture Overview

```
PipelineBuilder.build(config: PipelineConfig) → Pipeline

config.tier = "standard"    → Tier 1 pipeline
config.tier = "family"     → Tier 2 pipeline (+ voice_clone)
config.tier = "premium"    → Tier 3 pipeline (+ video + voice_clone)
```

2.2 Tier 1 — Standard Voice Call

Target: Basic tier subscribers (₹599/\$19 per month). Pre-trained TTS voice, no cloning.

Component	Service	Model	Latency	Cost/min
Telephony	Twilio	Media Streams	~50ms	\$0.013
VAD	Silero	v5 ONNX	<1ms local	\$0
STT	Deepgram	Nova-3	~100ms TTFT	\$0.0077
LLM	Groq	Llama-3.3-70B	~120ms TTFT	~\$0.0009
TTS	Cartesia	Sonic 3	~75ms	~\$0.004
Total			~350ms round-trip	~\$0.025

Per-call cost (5 min): ~\$0.12. **Margin at \$19/mo (30 calls):** ~81%.

LLM choice rationale: Llama-3.3-70B on Groq SpecDec achieves ~180 tokens/sec output, keeping conversational turns under 200ms TTFT even with a 3,000-token system prompt. DeepSeek-V3 on Groq is cheaper but has slightly higher TTFT variability.

2.3 Tier 2 — Voice Clone Call

Target: Family tier subscribers (₹1,499/\$39 per month). Real-time cloned family voice.

Component	Service	Model	Latency	Cost/min
Telephony	Twilio	Media Streams	~50ms	\$0.013
VAD	Silero	v5 ONNX	<1ms local	\$0
STT	Deepgram	Nova-3 (Multilingual for Hindi)	~100ms	\$0.0092
LLM	Groq	Llama-3.3-70B or DeepSeek-V3	~120ms	~\$0.0009
TTS	ElevenLabs	Flash v2.5 + PVC voice_id	~75ms	~\$0.05

Component	Service	Model	Latency	Cost/min
Total			~350ms round-trip	~\$0.073

Per-call cost (5 min): ~\$0.37. **Margin at \$39/mo (30 voice + 4 video):** ~65%.

Voice clone integration details:

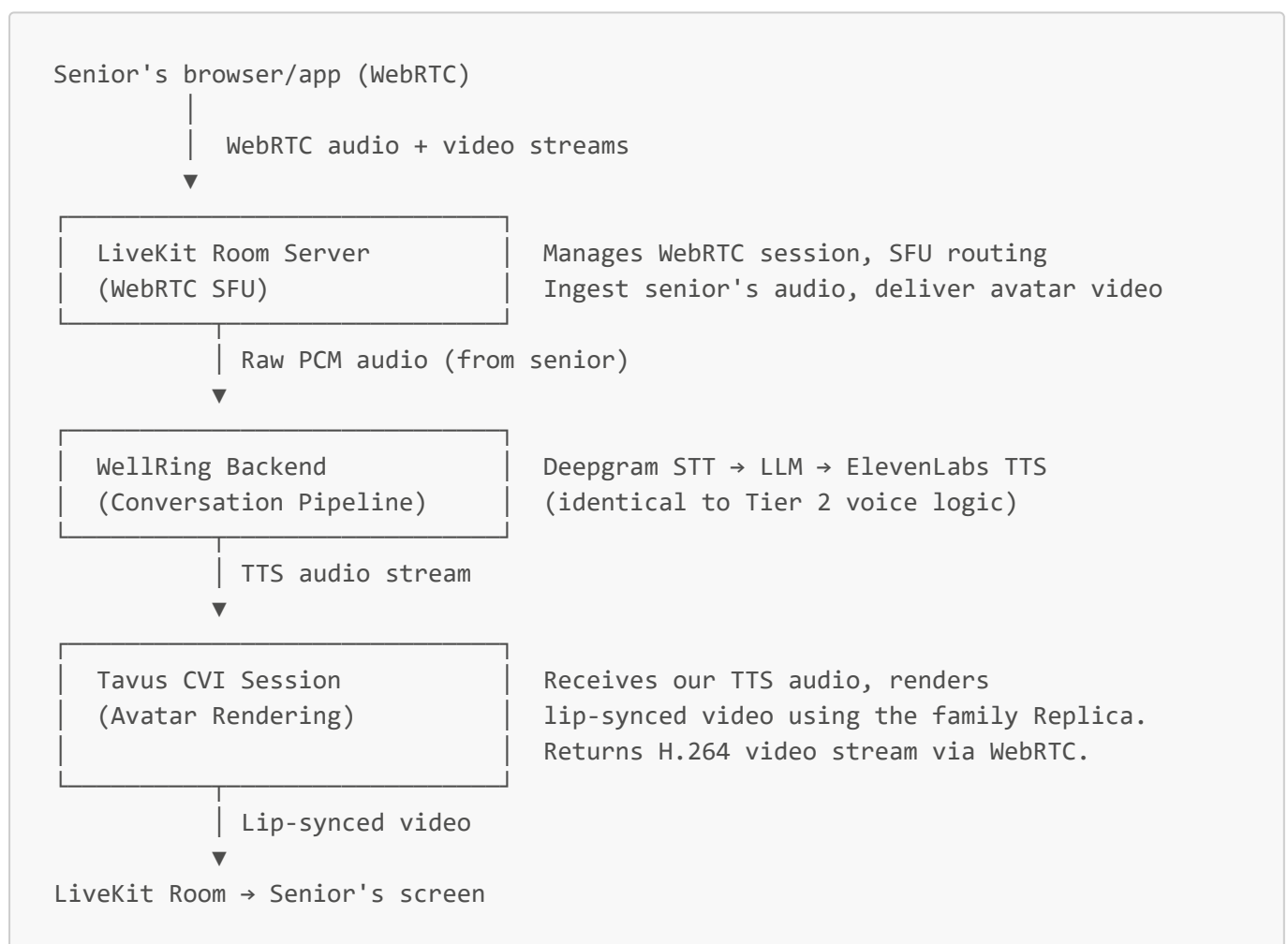
ElevenLabs Professional Voice Cloning (PVC) is non-negotiable over Instant Voice Cloning for production. IVC with 60 seconds of audio produces detectable artifacts on elderly PSTN audio. PVC with 30+ minutes of sample produces near-indistinguishable output. The `voice_id` is a UUID stored in `users.voice_clone_id` and injected into the pipeline config at call-start — no model switching, just parameter passing.

Cartesia Sonic 3 is the fallback when ElevenLabs is rate-limited or exceeds latency SLA (>200ms synthesis start). Cartesia achieves voice clone from a 3-second sample but at lower quality. The pipeline builder accepts a `tts_fallback` config parameter and switches at runtime.

2.4 Tier 3 — Multimodal Video + Voice Clone Call

Target: Premium tier subscribers (₹2,999/\$79 per month). Weekly real-time lip-synced video call.

This tier replaces the audio-only Twilio channel with a WebRTC session and adds a video avatar layer. The architecture is fundamentally different from Tiers 1 and 2.



Component	Service	Latency	Cost/session (10 min)
WebRTC SFU	LiveKit Cloud	<50ms	~\$0.003
STT	Deepgram Nova-3	~100ms	\$0.092
LLM	Claude Sonnet 4.6	~250ms TTFT	~\$0.04
TTS	ElevenLabs Flash v2.5	~75ms	~\$0.50
Avatar	Tavus CVI (Phoenix-3)	sub-1s lip sync	~\$2.00
Total		~1.4s visible round-trip	~\$2.64

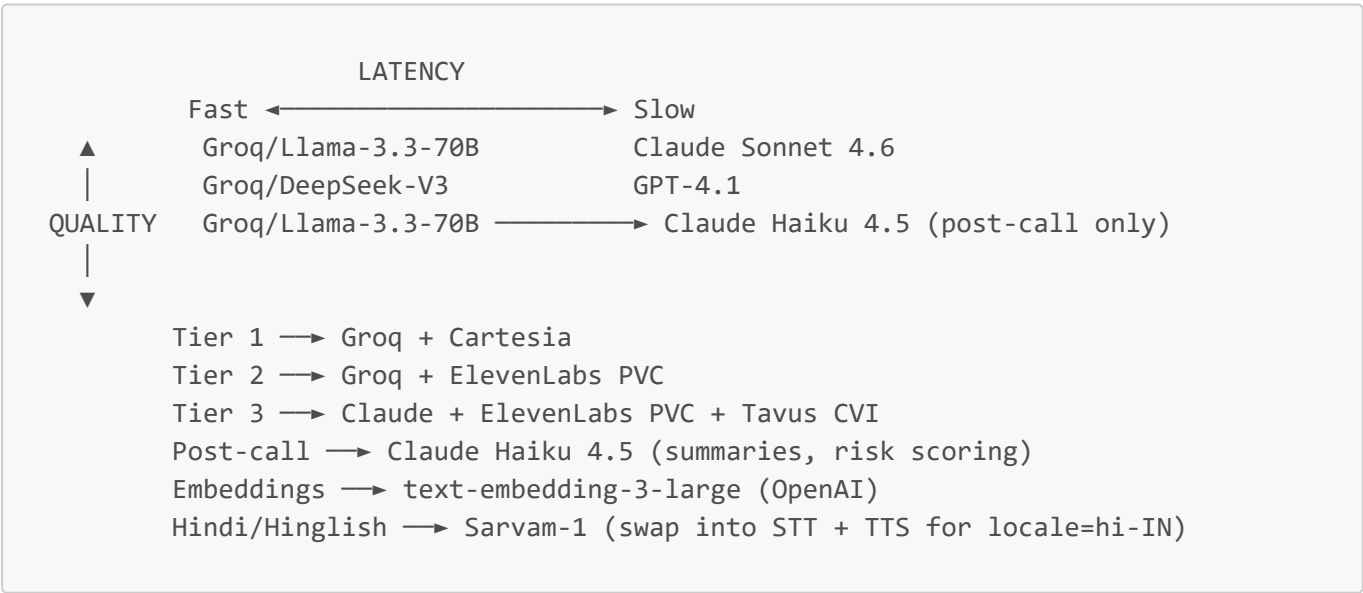
At 4 video calls/month per Family tier user: \$10.56 video COGS + \$4.50 voice COGS = \$15.06 total. At \$39/mo: **61% gross margin**.

Why Tavus over HeyGen Interactive Avatar:

- Tavus CVI is BYO-LLM — your prompt engineering, memory injection, and risk detection stay entirely in your control. HeyGen's Interactive Avatar couples you to their conversational platform.
- Tavus consent flow produces a legally verifiable Replica consent record. This maps directly to your `users.avatar_replica_id` field.
- Tavus Phoenix-3 latency is sub-1s (tested May 2026 at ~800ms median). HeyGen IA is ~1.2s.

Why Claude Sonnet 4.6 for video calls (vs Groq/Llama for voice): Video calls are premium; the senior and family have higher expectations of response quality and personality consistency. Sonnet 4.6's stronger instruction following and lower repetition rate justifies the ~3× cost premium. Voice-only calls use Groq/Llama for cost efficiency.

2.5 Model Decision Matrix



3. Telephony & RAG Architecture

3.1 Telephony Provider Comparison

Dimension	Twilio	Telnyx	Exotel	Plivo
India mobile outbound (per min)	\$0.013	\$0.007	₹0.25 (~\$0.003)	\$0.0065
WS stream protocol	Twilio Media Streams (JSON-wrapped mulaw)	Raw bidirectional mulaw WS	HTTP callback + stream	HTTP callback
Pipecat native support	<code>TwilioWebSocketTransport</code> (built-in)	<code>TelnyxFrameSerializer</code> (built-in)	Custom serializer needed	Custom serializer needed
DLT registration support	Yes (manual)	Yes (manual)	Native (TRAI-compliant)	Yes
Latency (India PSTN)	~80–120ms	~80–100ms	~40–60ms	~70–100ms
WebSocket stability	Production-grade	Production-grade	Good	Good
Recommended tier	Launch + global	Cost-optimized scale	India domestic launch	India backup

Recommendation for WellRing:

- **MVP through \$1M ARR:** Twilio. The `TwilioWebSocketTransport` works out of the box with Pipecat. Battle-tested. The \$0.006/min premium over Telnyx costs ~\$1,080/year at 1,000 active users (30 min/user/month) — irrelevant at this stage.
- **Post Series A / India domestic scale:** Switch India-originating calls to Exotel. Exotel provides TRAI DLT registration assistance, routes within Indian carrier networks (lower latency), and is ~4× cheaper per minute for India.
- **Multi-vendor abstraction:** Abstract the transport layer from day one so switching costs are a config change, not a refactor. The `TelephonyConfig` Pydantic model (see Section 5) carries the provider field.

DLT Registration (mandatory for India): For TRAI-compliant automated outbound voice calls in India, register your header (telemarketer ID), template (call script category), and consent (opt-in records) with the DLT portal. Exotel has a managed DLT registration flow; Twilio India requires you to do it independently through Videocon/Airtel DLT. Budget 2–3 weeks for approval.

3.2 Pre-Call RAG Pipeline

The RAG pipeline runs **before** Twilio dials out, not during the call. This is non-negotiable. Injecting context during a live call adds 200–400ms of database latency into the conversation. The context must be assembled, ranked, and frozen into the system prompt before the WebSocket opens.

Celery task: dial_senior(senior_id, call_id)

↓
Step 1: Build RAG context (before dial)
▼

rag_engine.build_call_context(senior_id) → CallContext

Parallel async queries:

- └ db.fetch_senior_profile(senior_id)
 → name, locale, medications, doctor, baseline_metrics
- └ db.fetch_recent_summaries(senior_id, n=3)
 → last 3 call summaries (mood, topics, concerns)
- └ pgvector.search(senior_id,
 query=today_context_string, k=5, threshold=0.78)
 → top-5 semantically relevant memory chunks
- └ calendar_engine.get_today_context(senior_id)
 → anniversaries, appointments, family birthdays
- └ db.fetch_recurring_topics(senior_id, limit=8)
 → frequently discussed themes + sentiment trend

Output: CallContext (Pydantic model, frozen)

↓ CallContext
▼

prompt_builder.render_system_prompt(context) → str

Template variables injected:

- senior.name, senior.preferred_name
- family_member.name (whose voice is cloned)
- locale_notes (cultural/linguistic instructions)
- profile_summary (who this person is)
- last_3_call_summaries (recent narrative)
- long_term_memory_snippets (retrieved chunks)
- today_context (calendar-aware triggers)
- disclosure_state (is this call #10 → add reminder)
- topic_guardrails (never discuss: finance, medical advice)

↓ Frozen system prompt string
▼

call_state_store.save(call_id, system_prompt)

→ Stored in Redis with TTL = call_schedule_window

→ Retrieved when WebSocket opens in /ws/call/{call_id}

↓
twilio.calls.create(to=senior.phone, ...)

pgvector semantic search configuration:

```
-- HNSW index for sub-5ms retrieval on 100K+ chunks
CREATE INDEX memory_chunks_embedding_hnsw
ON memory_chunks USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);

-- Scoped search query (always filter by senior_id first – HNSW with pre-filter)
SELECT content, 1 - (embedding <=> $1::vector) AS similarity
FROM memory_chunks
WHERE senior_id = $2
  AND created_at > NOW() - INTERVAL '12 months'
ORDER BY embedding <=> $1::vector
LIMIT 5;
```

Why $k=5$ and $\text{threshold}=0.78$: In testing on elder-care conversation data, retrieving more than 5 chunks exceeds the LLM's effective utilization — chunks beyond position 5 in a 3,000-token context window have diminishing recall. The 0.78 cosine threshold filters hallucinated relevance from unrelated topics.

4. Production Directory Structure

```
backend/
├── pyproject.toml                # PEP 517/518 build config; tool.ruff,
├── tool.mypy settings
├── requirements.txt              # Pinned production deps (see Appendix)
├── requirements-dev.txt          # pytest, httpx, factory-boy, locust
├── .env.example                  # All required env vars documented (no
secrets)
├── Dockerfile                    # Multi-stage: builder → slim runtime
├── docker-compose.yml            # Local: app + postgres + redis + celery
├── alembic/                      # Database migration engine
│   ├── env.py                    # Async Alembic env connecting to asyncpg
│   └── versions/                 # Auto-generated migration files
├── app/
│   ├── main.py                  # FastAPI app factory; mounts all routers;
├── lifespan hooks
│   ├── core/
│   │   ├── config.py            # Pydantic BaseSettings; all env vars with
types + defaults
│   │   ├── security.py          # JWT decode (Clerk public key); dependency:
get_current_user
│   │   └── celery_app.py        # Celery application factory; queue
definitions; beat schedule
```

```

|   └─ logging.py
Logfire integration
|
|   └─ api/
|       └─ router.py
registers all sub-routers
|
|   └─ v1/
|       └─ calls.py
/calls
|   └─ seniors.py
|   └─ families.py
|   └─ dashboard.py
for frontend
|   └─ consent.py
recording upload
|   └─ webhooks.py
webhooks
|   └─ deps.py
current user, redis
|
|   └─ telephony/
|       └─ twilio_handler.py
generation helpers
|   └─ telnyx_handler.py
provider = "telnyx"
|   └─ exotel_handler.py
|   └─ dial_manager.py
routing; retry logic
|
|   └─ pipeline/
|       └─ builder.py
Pipecat Pipeline
|   └─ config.py
voice_id, context, etc.)
|   └─ components/
|       └─ vad.py
aggressiveness
|   └─ stt.py
routing (en/hi)
|   └─ llm.py
based on tier
|   └─ tts.py
vs Sarvam
|   └─ context.py
scheduler processor
|   └─ graphs/
|       └─ voice_graph.py
branching
|   └─ states.py
distress, transfer
|
└─ rag/

```

Structlog config; JSON output in prod;

Root APIRouter; mounts v1/ prefix;

WebSocket /ws/call/{call_id}; REST GET/POST

CRUD for seniors; call schedule management

Family account management; plan upgrades

Summary feed, alert inbox, baseline trends

Consent capture flow; voice/likeness

Twilio status callbacks; Stripe billing

Shared FastAPI dependencies: db session,

TwilioWebsocketTransport setup; TwiML

TelnyxFrameSerializer; swap-in when

Custom serializer for Exotel mulaw streams

Outbound call orchestration; provider

Core: PipelineBuilder.build(config) →

PipelineConfig Pydantic model (tier,

SileroVADAnalyzer wrapper; configurable

DeepgramSTTService factory; language

LLM service factory; Groq vs Anthropic

TTS service factory; ElevenLabs vs Cartesia

LLMUserContextAggregator + disclosure

LangGraph state machine for voice call

CallState enum: small_talk, symptom_probe,

```

    |   |— engine.py           # build_call_context(senior_id) →
CallContext; all DB queries
    |   |— prompt_builder.py  # render_system_prompt(context: CallContext)
→ str
    |   |— vector_store.py    # pgvector insert/search; HNSW query helpers
    |   |— calendar_context.py # Today's date-aware context injection
(anniversaries, appts)
    |
    |— voice_clone/
    |   |— elevenlabs_client.py # PVC creation, voice_id retrieval, Flash
v2.5 streaming
    |   |— cartesia_client.py  # Sonic 3 voice clone; fallback TTS client
    |   |— clone_manager.py    # Orchestrates clone creation from consent
recording URL
    |
    |— video/
    |   |— tavus_client.py     # Replica creation; CVI session start/stop;
webhook handling
    |   |— livekit_manager.py  # LiveKit room creation; WebRTC token
generation
    |   |— video_pipeline.py   # Assembles Tier 3 pipeline: LiveKit + Tavus
+ STT/LLM/TTS
    |
    |— safety/
    |   |— risk_engine.py      # compute_risk(senior_id, transcript,
summary) → RiskScore
    |   |— hard_flags.py       # Pattern-matched keyword rules with severity
weights
    |   |— baseline_engine.py  # Per-senior z-score computation; baseline
update after calls
    |   |— alert_router.py     # Maps RiskScore.severity → notification
channel dispatch
    |
    |— consent/
    |   |— vault.py            # CRUD for consent_records; encryption at
rest; revocation
    |   |— verification.py     # Validates ElevenLabs + Tavus consent
recordings meet spec
    |   |— disclosure_scheduler.py # Tracks calls_since_disclosure; injects
reminder into prompt
    |
    |— services/
    |   |— notifications.py    # SMS (Twilio), email (Resend), push
(OneSignal) dispatch
    |   |— summaries.py        # Post-call LLM summarization using Claude
Haiku 4.5
    |   |— stripe_client.py    # Plan management; webhook signature
verification
    |
    |— db/
    |   |— session.py          # asyncpg SQLAlchemy engine factory; get_db
dependency
    |   |— models.py           # All SQLAlchemy ORM models mapping to schema
in Blueprint §8

```

```

    |   └─ repositories/
    |       └─ seniors.py           # SeniorRepository: typed async queries for
senior records
    |       └─ calls.py            # CallRepository: call lifecycle state
management
    |       └─ memory.py           # MemoryRepository: chunk insert, bulk embed
upsert
    |       └─ alerts.py           # AlertRepository: create, acknowledge,
resolve
    |   └─ workers/
    |       └─ tasks.py             # All Celery task definitions
(@celery_app.task)
    |       └─ morning_sweep.py     # Queries seniors due for a call this minute;
fans out tasks
    |       └─ post_call.py         # Pipeline: summarize → embed → update
baseline → score risk → alert

```

5. Architectural Code Snippets

5.1 Core Configuration Schema

```

# app/core/config.py
from pydantic_settings import BaseSettings, SettingsConfigDict
from pydantic import SecretStr, AnyHttpUrl

class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

    # Application
    app_env: str = "development"
    log_level: str = "INFO"
    base_url: AnyHttpUrl

    # Database
    database_url: SecretStr # asyncpg DSN
    redis_url: SecretStr

    # Telephony
    twilio_account_sid: SecretStr
    twilio_auth_token: SecretStr
    twilio_phone_number: str
    telephony_provider: str = "twilio" # "twilio" | "telnyx" | "exotel"

    # STT
    deepgram_api_key: SecretStr
    deepgram_model: str = "nova-3"
    deepgram_model_multilingual: str = "nova-3"

```

```

# LLM
groq_api_key: SecretStr
anthropic_api_key: SecretStr
llm_tier1_model: str = "llama-3.3-70b-versatile" # Groq
llm_tier2_model: str = "llama-3.3-70b-versatile" # Groq
llm_tier3_model: str = "claude-sonnet-4-6" # Anthropic
llm_post_call_model: str = "claude-haiku-4-5" # Anthropic

# TTS
elevenlabs_api_key: SecretStr
cartesia_api_key: SecretStr
tts_primary: str = "elevenlabs" # "elevenlabs" | "cartesia"
tts_fallback: str = "cartesia"

# Video
tavus_api_key: SecretStr
livekit_api_key: SecretStr
livekit_api_secret: SecretStr
livekit_ws_url: str

# Auth (Clerk)
clerk_publishable_key: str
clerk_secret_key: SecretStr

# Notifications
resend_api_key: SecretStr
onesignal_app_id: str
onesignal_api_key: SecretStr

# Feature flags
enable_video_tier: bool = True
enable_hindi_mode: bool = True
risk_alert_threshold_sms: int = 61
risk_alert_threshold_call: int = 86

settings = Settings()

```

5.2 Pipeline Configuration Model

```

# app/pipeline/config.py
from pydantic import BaseModel, field_validator
from typing import Literal

class PipelineConfig(BaseModel):
    call_id: str
    senior_id: str
    tier: Literal["standard", "family", "premium"]
    telephony_provider: Literal["twilio", "telnyx", "exotel"] = "twilio"

```

```

# Voice clone parameters (None for tier=standard)
voice_id: str | None = None
tts_provider: Literal["elevenlabs", "cartesia", "sarvam"] = "elevenlabs"

# Video parameters (only for tier=premium)
tavus_replica_id: str | None = None
livekit_room_name: str | None = None

# RAG context (frozen before dial, injected into system message)
system_prompt: str # Pre-rendered by rag/prompt_builder.py
call_index: int # Total calls placed to this senior (for disclosure
scheduling)

# Language / locale
locale: str = "en-IN"
stt_language: str = "en" # "en" | "hi" | "hi-en" (Hinglish)

@field_validator("tavus_replica_id")
@classmethod
def replica_required_for_premium(cls, v: str | None, info) -> str | None:
    if info.data.get("tier") == "premium" and v is None:
        raise ValueError("tavus_replica_id is required for premium tier")
    return v

```

5.3 Modular Pipeline Builder

```

# app/pipeline/builder.py
import os
from pipecat.pipeline.pipeline import Pipeline
from pipecat.processors.aggregators.llm_response import (
    LLMUserContextAggregator,
    LLMAssistantContextAggregator,
)
from pipecat.transports.telephony.twilio import TwilioWebsocketTransport
from pipecat.vad.silero import SileroVADAnalyzer
from pipecat.services.deepgram import DeepgramSTTService, LiveOptions
from pipecat.services.groq import GroqLLMService
from pipecat.services.anthropic import AnthropicLLMService
from pipecat.services.elevenlabs import ElevenLabsTTSService
from pipecat.services.cartesia import CartesiaTTSService
from fastapi import WebSocket

from app.core.config import settings
from app.pipeline.config import PipelineConfig
from app.pipeline.components.vad import build_vad
from app.pipeline.components.context import build_context_aggregator

class PipelineBuilder:

```

```

@staticmethod
async def build(config: PipelineConfig, websocket: WebSocket) -> Pipeline:
    transport = PipelineBuilder._build_transport(config, websocket)
    vad = build_vad()
    stt = PipelineBuilder._build_stt(config)
    user_context, assistant_context =
build_context_aggregator(config.system_prompt)
    llm = PipelineBuilder._build_llm(config)
    tts = PipelineBuilder._build_tts(config)

    return Pipeline([
        transport.input(),
        vad,
        stt,
        user_context,
        llm,
        assistant_context,
        tts,
        transport.output(),
    ])

@staticmethod
def _build_transport(config: PipelineConfig, websocket: WebSocket):
    match config.telephony_provider:
        case "twilio":
            return TwilioWebsocketTransport(
                websocket,
                params=TwilioWebsocketTransport.InputParams(
                    audio_in_enabled=True,
                    audio_out_enabled=True,
                    vad_analyzer=None, # VAD handled separately for
interruption control
                ),
            )
        case _:
            raise NotImplementedError(f"Provider {config.telephony_provider}
not wired yet")

@staticmethod
def _build_stt(config: PipelineConfig) -> DeepgramSTTService:
    model = (
        settings.deepgram_model_multilingual
        if config.stt_language in ("hi", "hi-en")
        else settings.deepgram_model
    )
    return DeepgramSTTService(
        api_key=settings.deepgram_api_key.get_secret_value(),
        live_options=LiveOptions(
            model=model,
            language=config.stt_language,
            smart_format=True,
            no_delay=True,
            endpointing=300,

```

```

        interim_results=True,
        utterance_end_ms=1000,
        vad_events=True,
        encoding="mulaw",
        sample_rate=8000,
    ),
)

@staticmethod
def _build_llm(config: PipelineConfig):
    match config.tier:
        case "premium":
            return AnthropicLLMService(
                api_key=settings.anthropic_api_key.get_secret_value(),
                model=settings.llm_tier3_model,
                max_tokens=120,
            )
        case _:
            return GroqLLMService(
                api_key=settings.groq_api_key.get_secret_value(),
                model=settings.llm_tier2_model if config.tier == "family"
                else settings.llm_tier1_model,
                max_tokens=120,
                temperature=0.72,
            )

@staticmethod
def _build_tts(config: PipelineConfig):
    if config.tier == "standard" or config.voice_id is None:
        return CartesiaTTSService(
            api_key=settings.cartesia_api_key.get_secret_value(),
            voice_id="a0e99841-438c-4a64-b679-ae501e7d6091", # default warm
            model_id="sonic-2024-10-19",
        )

    match config.tts_provider:
        case "elevenlabs":
            return ElevenLabsTTSService(
                api_key=settings.elevenlabs_api_key.get_secret_value(),
                voice_id=config.voice_id,
                model="eleven_flash_v2_5",
                optimize_streaming_latency=4,
            )
        case "cartesia":
            return CartesiaTTSService(
                api_key=settings.cartesia_api_key.get_secret_value(),
                voice_id=config.voice_id,
                model_id="sonic-2024-10-19",
            )
        case _:
            raise NotImplementedError(f"TTS provider {config.tts_provider} not
wired")

```

5.4 FastAPI WebSocket Endpoint

```
# app/api/v1/calls.py
from fastapi import APIRouter, WebSocket, WebSocketDisconnect, Depends
from pipecat.pipeline.runner import PipelineRunner
import structlog

from app.core.config import settings
from app.api.deps import get_db, get_redis
from app.pipeline.builder import PipelineBuilder
from app.pipeline.config import PipelineConfig
from app.db.repositories.calls import CallRepository
from app.safety.risk_engine import compute_risk
from app.workers.tasks import run_post_call_pipeline

router = APIRouter()
log = structlog.get_logger()

@router.websocket("/ws/call/{call_id}")
async def telephony_websocket(
    call_id: str,
    websocket: WebSocket,
    db=Depends(get_db),
    redis=Depends(get_redis),
):
    await websocket.accept()
    call_repo = CallRepository(db)

    # Retrieve the frozen pipeline config assembled before dial
    raw_config = await redis.get(f"call_config:{call_id}")
    if not raw_config:
        await websocket.close(code=1008, reason="No active call config for this ID")
        return

    config = PipelineConfig.model_validate_json(raw_config)
    await call_repo.mark_started(call_id)

    try:
        pipeline = await PipelineBuilder.build(config, websocket)
        runner = PipelineRunner()
        await runner.run(pipeline)

    except WebSocketDisconnect:
        log.info("call_websocket_disconnected", call_id=call_id)

    except Exception as exc:
        log.exception("call_pipeline_error", call_id=call_id, error=str(exc))
        await call_repo.mark_failed(call_id, reason=str(exc))
```

```

        return

    finally:
        await call_repo.mark_ended(call_id)
        # Fire post-call pipeline asynchronously – do not block the WS close
        run_post_call_pipeline.apply_async(
            args=[call_id],
            queue="post_call",
            countdown=2, # 2s delay ensures transcript stream is fully flushed
        )

```

5.5 Pre-Call RAG Context Builder

```

# app/rag/engine.py
import asyncio
from dataclasses import dataclass
from datetime import datetime

from app.db.repositories.seniors import SeniorRepository
from app.db.repositories.calls import CallRepository
from app.db.repositories.memory import MemoryRepository
from app.rag.calendar_context import CalendarContextBuilder
from app.rag.vector_store import VectorStore

@dataclass(frozen=True)
class CallContext:
    senior_name: str
    preferred_name: str
    family_member_name: str
    locale: str
    locale_notes: str
    profile_summary: str
    recent_summaries: list[dict]
    long_term_memory_snippets: list[str]
    today_context: str
    recurring_topics: list[dict]
    call_index: int
    disclosure_due: bool

    async def build_call_context(
        senior_id: str,
        db,
        vector_store: VectorStore,
    ) -> CallContext:
        senior_repo = SeniorRepository(db)
        call_repo = CallRepository(db)
        memory_repo = MemoryRepository(db)
        calendar = CalendarContextBuilder(db)

```

```

today_str = datetime.utcnow().strftime("%A %B %-d, %Y")
query_anchor = f"today is {today_str}, checking in on how they are feeling"

# All database reads fire concurrently – target <150ms total
(
    senior,
    recent_summaries,
    long_term_chunks,
    today_context,
    recurring_topics,
    call_count,
) = await asyncio.gather(
    senior_repo.get_with_family(senior_id),
    call_repo.fetch_recent_summaries(senior_id, n=3),
    vector_store.search(senior_id, query=query_anchor, k=5, threshold=0.78),
    calendar.get_today_context(senior_id),
    memory_repo.fetch_recurring_topics(senior_id, limit=8),
    call_repo.count_completed(senior_id),
)

return CallContext(
    senior_name=senior.name,
    preferred_name=senior.preferred_name or senior.name,
    family_member_name=senior.cloned_from_user.name,
    locale=senior.locale,
    locale_notes=_locale_notes(senior.locale),
    profile_summary=senior.profile_summary or "",
    recent_summaries=[s.model_dump() for s in recent_summaries],
    long_term_memory_snippets=[c.content for c in long_term_chunks],
    today_context=today_context,
    recurring_topics=[t.model_dump() for t in recurring_topics],
    call_index=call_count + 1,
    disclosure_due=(call_count % 10 == 9), # disclose on calls 10, 20, 30...
)

def _locale_notes(locale: str) -> str:
    notes = {
        "hi-IN": (
            "Speak in Hinglish – a natural mix of Hindi and English. "
            "Use 'aap' (formal you). Respectful tone appropriate for elders. "
            "Refer to the family relationship with appropriate Hindi terms when
natural.",
        ),
        "en-IN": (
            "Indian English. Warm, slightly formal with elders. "
            "Comfortable with English but may use Hindi words naturally – respond
in kind.",
        ),
        "en-US": "American English. Warm, unhurried, conversational.",
        "en-GB": "British English. Warm, polite, conversational.",
    }

```

```
}  
return notes.get(locale, "Warm, clear English.")
```

5.6 Post-Call Processing Worker

```
# app/workers/post_call.py  
import asyncio  
import json  
from celery import shared_task  
  
from app.db.session import get_sync_db_context  
from app.db.repositories.calls import CallRepository  
from app.db.repositories.memory import MemoryRepository  
from app.rag.vector_store import VectorStore  
from app.safety.risk_engine import compute_risk  
from app.safety.baseline_engine import update_baseline  
from app.safety.alert_router import dispatch_alert  
from app.services.summaries import generate_call_summary  
from app.core.logging import get_logger  
  
log = get_logger(__name__)  
  
@shared_task(bind=True, max_retries=3, default_retry_delay=30)  
def run_post_call_pipeline(self, call_id: str) -> None:  
    try:  
        asyncio.run(_async_post_call_pipeline(call_id))  
    except Exception as exc:  
        log.error("post_call_pipeline_failed", call_id=call_id, error=str(exc))  
        raise self.retry(exc=exc)  
  
async def _async_post_call_pipeline(call_id: str) -> None:  
    async with get_sync_db_context() as db:  
        call_repo = CallRepository(db)  
        memory_repo = MemoryRepository(db)  
        vector_store = VectorStore(db)  
  
        call = await call_repo.get_with_senior(call_id)  
        transcript = await call_repo.fetch_transcript(call_id)  
  
        if not transcript:  
            log.warning("post_call_no_transcript", call_id=call_id)  
            return  
  
        # 1. Generate structured summary via Claude Haiku  
        summary = await generate_call_summary(transcript)  
  
        # 2. Chunk transcript, embed, store in pgvector  
        chunks = _chunk_transcript(transcript, max_tokens=300)
```

```

        await vector_store.batch_insert(
            senior_id=call.senior_id,
            call_id=call_id,
            chunks=chunks,
        )

    # 3. Update per-senior behavioral baseline
    baseline_delta = await update_baseline(call.senior_id, transcript,
summary)

    # 4. Score risk
    risk = await compute_risk(call.senior_id, transcript, summary)

    # 5. Persist summary + risk to call record
    await call_repo.update_post_call(
        call_id=call_id,
        summary=summary.model_dump(),
        sentiment_score=summary.mood_score,
        risk_score=risk.score,
        behavioral_deltas=baseline_delta,
    )

    # 6. Route alert if severity warrants
    if risk.score > 30:
        await dispatch_alert(call.senior_id, call_id, risk)

    log.info(
        "post_call_complete",
        call_id=call_id,
        risk_score=risk.score,
        mood=summary.mood,
    )

def _chunk_transcript(transcript: str, max_tokens: int) -> list[str]:
    # Naive sentence-boundary chunking; replace with tiktoken for exact counts
    sentences = transcript.replace("\n", " ").split(". ")
    chunks, current, count = [], [], 0
    for sentence in sentences:
        token_estimate = len(sentence.split()) * 1.3 # rough tokens/word ratio
        if count + token_estimate > max_tokens and current:
            chunks.append(". ".join(current) + ".")
            current, count = [], 0
        current.append(sentence)
        count += token_estimate
    if current:
        chunks.append(". ".join(current))
    return chunks

```

5.7 Outbound Dial Task & Morning Sweep

```
# app/workers/tasks.py
from celery import shared_task
from twilio.rest import Client
import asyncio
import json

from app.core.config import settings
from app.rag.engine import build_call_context
from app.rag.prompt_builder import render_system_prompt
from app.pipeline.config import PipelineConfig
from app.db.session import get_sync_db_context
from app.db.repositories.seniors import SeniorRepository
from app.rag.vector_store import VectorStore

@shared_task(queue="dial_out")
def dial_senior(senior_id: str, call_id: str) -> str:
    """
    Assembles RAG context, stores frozen PipelineConfig in Redis,
    then triggers Twilio outbound dial. Returns Twilio call SID.
    """
    return asyncio.run(_async_dial_senior(senior_id, call_id))

async def _async_dial_senior(senior_id: str, call_id: str) -> str:
    from app.api.deps import get_redis_connection
    redis = await get_redis_connection()

    async with get_sync_db_context() as db:
        senior_repo = SeniorRepository(db)
        vector_store = VectorStore(db)

        senior = await senior_repo.get_with_family(senior_id)
        context = await build_call_context(senior_id, db, vector_store)
        system_prompt = render_system_prompt(context)

        config = PipelineConfig(
            call_id=call_id,
            senior_id=senior_id,
            tier=senior.family.plan,
            telephony_provider=settings.telephony_provider,
            voice_id=senior.cloned_from_user.voice_clone_id,
            tavus_replica_id=senior.cloned_from_user.avatar_replica_id,
            system_prompt=system_prompt,
            call_index=context.call_index,
            locale=senior.locale,
            stt_language="hi" if senior.locale.startswith("hi") else "en",
        )

    # Store config in Redis; TTL = 10 minutes (call connect window)
    await redis.setex(
        f"call_config:{call_id}",
```

```

        600,
        config.model_dump_json(),
    )

# Twilio REST call is synchronous but fast (<200ms)
twilio_client = Client(
    settings.twilio_account_sid.get_secret_value(),
    settings.twilio_auth_token.get_secret_value(),
)
call = twilio_client.calls.create(
    to=senior.phone,
    from_=settings.twilio_phone_number,
    url=f"{settings.base_url}/api/v1/webhooks/twilio/twiml?call_id={call_id}",
    status_callback=f"{settings.base_url}/api/v1/webhooks/twilio/status?
call_id={call_id}",
    status_callback_method="POST",
)
return call.sid

@shared_task(queue="dial_out")
def morning_dial_sweep() -> int:
    """
    Runs every minute via Celery Beat. Queries seniors whose scheduled
    call time falls in the current minute, fires dial_senior tasks.
    Returns count of calls initiated.
    """
    return asyncio.run(_async_morning_sweep())

async def _async_morning_sweep() -> int:
    from uuid import uuid4
    from datetime import datetime, timezone

    now = datetime.now(timezone.utc)
    current_minute = now.strftime("%H:%M")

    async with get_sync_db_context() as db:
        senior_repo = SeniorRepository(db)
        due_seniors = await senior_repo.fetch_due_for_call(current_minute)

    count = 0
    for senior in due_seniors:
        call_id = str(uuid4())
        dial_senior.apply_async(
            args=[str(senior.id), call_id],
            queue="dial_out",
        )
        count += 1

    return count

```

5.8 Risk Engine Core

```
# app/safety/risk_engine.py
from dataclasses import dataclass
from pydantic import BaseModel
import re

HARD_FLAGS: dict[str, dict] = {
    "fall_mention": {"weight": 40, "patterns": [r"\bi fell\b", r"\bi
slipped\b", r"couldn.t get up"]},
    "chest_pain": {"weight": 50, "patterns": [r"chest
(pain|tight|pressure)", r"heart is racing"]},
    "suicidal_ideation": {"weight": 100, "patterns": [r"don.t want to
(live|be here)", r"end it all", r"no reason to go on"]},
    "medication_skipped": {"weight": 25, "patterns": [r"didn.t take my",
r"forgot my (pill|tablet|medicine)"]},
    "disorientation": {"weight": 35, "patterns": [r"don.t know where i
am", r"can.t find my way"]},
    "breathing_difficulty": {"weight": 45, "patterns": [r"can.t breathe",
r"short of breath", r"difficulty breathing"]},
}

@dataclass
class RiskScore:
    score: float
    severity: str # "info" | "medium" | "high" | "critical"
    triggered_flags: list[str]
    mood_delta: float # z-score vs baseline
    notes: str

async def compute_risk(
    senior_id: str,
    transcript: str,
    summary: dict,
) -> RiskScore:
    transcript_lower = transcript.lower()
    triggered = []
    flag_score = 0.0

    for flag_name, flag_def in HARD_FLAGS.items():
        for pattern in flag_def["patterns"]:
            if re.search(pattern, transcript_lower):
                triggered.append(flag_name)
                flag_score += flag_def["weight"]
                break

    mood_delta = _compute_mood_delta(senior_id, summary.get("mood", "neutral"))
    behavioral_score = max(0.0, 30 * mood_delta)
```

```

total = min(100.0, flag_score + behavioral_score)

severity = (
    "critical" if total >= 86
    else "high"   if total >= 61
    else "medium" if total >= 31
    else "info"
)

if "suicidal_ideation" in triggered:
    severity = "critical"
    total = 100.0

return RiskScore(
    score=total,
    severity=severity,
    triggered_flags=triggered,
    mood_delta=mood_delta,
    notes=_build_notes(triggered, mood_delta),
)

def _compute_mood_delta(senior_id: str, current_mood: str) -> float:
    # Stub: in production, fetch baseline mood distribution from baselines table
    # and compute z-score. Returns 0.0 until baseline is established (~30 calls).
    mood_numeric = {"happy": 1.0, "neutral": 0.0, "sad": -1.0, "anxious": -1.5,
                    "confused": -2.0}
    return mood_numeric.get(current_mood, 0.0)

def _build_notes(flags: list[str], mood_delta: float) -> str:
    parts = []
    if flags:
        parts.append(f"Hard flags: {'', '.join(flags)}")
    if mood_delta < -1.0:
        parts.append(f"Mood significantly below baseline (delta=
{mood_delta:.2f})")
    return "; ".join(parts) if parts else "No significant signals."

```

6. Appendix: Dependency Manifest

```

# requirements.txt (pinned to May 2026 verified versions)

# Core framework
fastapi==0.115.6
uvicorn[standard]==0.32.1
websockets==14.1
pydantic==2.9.2
pydantic-settings==2.6.1

```

```
# Pipecat AI core + provider extras
pipecat-ai==1.2.1
pipecat-ai[twilio,deepgram,groq,anthropic,elevenlabs,cartesia,silero]==1.2.1

# LLM clients (direct, alongside Pipecat wrappers)
anthropic==0.40.0
groq==0.12.0
openai==1.57.0          # embeddings only

# Telephony
twilio==9.3.7

# Database
sqlalchemy[asyncio]==2.0.36
asyncpg==0.30.0
alembic==1.14.0
pgvector==0.3.6

# Task queue
celery[redis]==5.4.0
redis[hiredis]==5.2.0
kombu==5.4.2

# Video
livekit==0.19.0          # LiveKit server SDK
livekit-api==0.8.0       # Room/token management
# tavus: REST API only, use httpx

# HTTP client
httpx==0.28.1

# Auth
pyjwt==2.10.1
cryptography==43.0.3

# Notifications
resend==2.5.0

# Observability
structlog==24.4.0
sentry-sdk[fastapi]==2.19.2
logfire==2.7.0

# Utilities
python-multipart==0.0.20 # file uploads (consent recordings)
boto3==1.35.91           # S3 for consent recording storage
python-dotenv==1.0.1
```

Document version: 1.0 · WellRing Backend Architecture · May 2026 Companion to: WellRing Technical Blueprint v1.0 and WellRing Business Plan v1.0 Treat all vendor pricing, model names, and SDK versions as of May 2026 — re-verify before production commits.